

```
$ prisma --production
```

Prisma ORM กับ Node.js

การจัดการ Migration — Production Ready

Type-safe ORM

Schema-first

Safe Migration

สำหรับ Backend Developer ที่ทำงานกับ SQL Server / MySQL ในองค์กรจริง

Beginner → Intermediate • Node.js + Express.js

Prisma คืออะไร

- Prisma คือ ORM (Object Relational Mapper) ยุคใหม่สำหรับ
- ช่วยลดการเขียน SQL ด้วยตัวเอง — ใช้ API ของ Prisma Client
- ใช้ `schema.prisma` เป็น single source of truth ของโครงสร้าง
- รองรับ PostgreSQL, MySQL, SQL Server, SQLite, MongoDB
- มีเครื่องมือครบชุด: Client, Migrate, Studio, Introspection

ORM

`schema.prisma`



TypeScript API



Database

ทำไมต้องใช้ Prisma

Type-safe

auto-complete + compile-time check

Query ง่าย

API เป็น object ไม่ต้องเขียน SQL

ลด Bug

ตรวจสอบชื่อ field / type ได้ทันที

Migration

version-control ของ schema

ภาพรวมการทำงาน (API Flow)



Data Request → Response

schema.prisma กำหนดโครงสร้างข้อมูลระหว่าง Prisma Client ↔ Database

การติดตั้ง Prisma

- ต้องมี Node.js ≥ 18 และโปรเจกต์ Node.js แล้ว
- ติดตั้ง Prisma CLI ไว้เป็น dev dependency
- ติดตั้ง Prisma Client สำหรับใช้งานใน runtime จริง



bash

```
# ติดตั้ง Prisma CLI (dev)
npm install prisma --save-dev

# ติดตั้ง Prisma Client
npm install @prisma/client
```

เริ่มต้นใช้งาน (npx prisma init)



bash

```
# สร้างโครงสร้าง Prisma เริ่มต้น  
npx prisma init
```

โครงสร้างที่ถูกสร้างขึ้น



tree

```
my-app/  
├── prisma/  
│   └── schema.prisma # กำหนด model / datasource  
├── .env               # เก็บ DATABASE_URL  
└── package.json
```

ตั้งค่า Database (.env)

- ใส่ connection string ไว้ในไฟล์ .env เท่านั้น
- ห้าม commit .env ขึ้น git — ใช้ .env.example แทน
- รูปแบบ URL ต่างกันตาม database ที่ใช้



.env

```
# MySQL
DATABASE_URL="mysql://user:pass@localhost:3306/mydb"

# SQL Server
DATABASE_URL="sqlserver://localhost:1433;database=mydb;user=sa;password=..."

# PostgreSQL
DATABASE_URL="postgresql://user:pass@localhost:5432/mydb"
```

Prisma Schema (model / field / type)

- model = หนึ่งตารางในฐานข้อมูล
- แต่ละ field ประกอบด้วย: ชื่อ + type + modifier
- attribute เช่น @id, @unique, @default กำหนดคุณสมบัติ field



prisma

```
model User {  
  id      Int      @id @default(autoincrement()) // รหัสหลัก  
  name    String                               // ชื่อผู้ใช้  
  email   String  @unique                       // ห้ามซ้ำ  
}
```

Prisma Client

- Prisma Client ถูก generate จาก schema.prisma อัตโนมัติ
- ควรสร้าง instance เดียวทั้งแอป (singleton) เพื่อไม่เปลืองการเชื่อมต่อ
- ใช้ await กับทุก query เพราะเป็น Promise-based



javascript

```
// สร้าง instance ของ Prisma Client หนึ่งครั้งต่อแอป
const { PrismaClient } = require('@prisma/client')
const prisma = new PrismaClient()

module.exports = prisma
```

Create — เพิ่มข้อมูลใหม่

- ใช้ `prisma.<model>.create()` พร้อมส่ง data เป็น object

```
// สร้างผู้ใช้ใหม่ 1 record
await prisma.user.create({
  data: {
    name: "John",
    email: "john@mail.com",
  }
})
```

javascript

Read — อ่านข้อมูล

- `findMany()` — ดึงหลายรายการ
- `findUnique()` — ดึงรายการเดียวด้วย field ที่เป็น unique
- `findFirst()` — ดึงรายการแรกโดยตรงเงื่อนไข



javascript

```
// ดึงผู้ใช้ทั้งหมด
const users = await prisma.user.findMany()

// ดึงผู้ใช้คนเดียวตาม email
const user = await prisma.user.findUnique({
  where: { email: "john@mail.com" }
})
```

Update — แก้ไขข้อมูล

- `update()` ต้องระบุ `where` ด้วย field ที่เป็น unique
- `updateMany()` ใช้แก้หลาย record พร้อมกันด้วยเงื่อนไข

javascript

```
// แก้ชื่อผู้ใช้นี้
await prisma.user.update({
  where: { id: 1 },
  data: { name: "John Doe" }
})
```

Delete — ลบข้อมูล

- delete() ระบุ where ด้วย unique field เช่น id
- deleteMany() ลบหลาย record ตามเงื่อนไข
- ระวัง cascade — ตั้งค่าใน schema ถ้ามี relation



javascript

```
// ลบผู้ใช้หนึ่งคน
await prisma.user.delete({
  where: { id: 1 }
})
```

Where / Filter — เงื่อนไข Query

- ใช้ operator เช่น equals, contains, gt, lt, in
- ใช้ AND / OR / NOT ต่อเงื่อนไขหลายชั้นได้

javascript

```
// ค้นหาผู้ใช้ที่มีอีเมลคำว่า gmail และ id > 10
await prisma.user.findMany({
  where: {
    email: { contains: "gmail" },
    id:    { gt: 10 }
  }
})
```

Select / Include — เลือก Field และดึง Relation

- select — เลือกเฉพาะ field ที่ต้องการ
- include — ดึงข้อมูลที่เป็น relation มาด้วย

javascript

```
// เลือกเฉพาะ id กับ email + ดึงโพสต์ทั้งหมดของ user
await prisma.user.findMany({
  select: { id: true, email: true },
  include: { posts: true }
})
```

Relation — ความสัมพันธ์ระหว่าง Model

- Post มี userId ชี้ไป User — One-to-Many
- @relation ระบุ field ที่ใช้ join



prisma

```
model Post {  
  id      Int  
  title   String  
  user    User    @relation(fields: [userId], references: [id])  
  userId  Int  
}
```

Query Relation — ดึงข้อมูลพร้อมความสัมพันธ์

- ใช้ include เพื่อ join ข้อมูลของ User เข้ามากับ Post
- ผลลัพธ์จะมี field user: {...} อยู่ในแต่ละ post



javascript

```
// ดึงโพสต์ทั้งหมด พร้อมข้อมูล user ของแต่ละโพสต์
await prisma.post.findMany({
  include: { user: true }
})
```

Migration คืออะไร

- กระบวนการอัปเดตโครงสร้างฐานข้อมูลให้ตรงกับ `schema.prisma`
- เหมือน version-control สำหรับ database (เทียบกับ git สำหรับ code)
- เก็บประวัติทุกการเปลี่ยนแปลงไว้ในโฟลเดอร์ `prisma/migrations/`
- ทำงานร่วมกันได้หลายคน โดยไม่ชน schema กันเอง
- แยกขั้นตอน dev (สร้าง/ทดสอบ) ออกจาก production (deploy จริง)

prisma migrate dev — ใช้ตอนพัฒนา

- สร้างไฟล์ migration จาก diff ของ schema.prisma
- รัน migration นั้นกับ database ทันที
- generate Prisma Client ใหม่อัตโนมัติ



bash

```
# สร้าง migration ชื่อ init และอัปเดต database  
npx prisma migrate dev --name init
```

Prisma Migration Tools — 4 คำสั่งหลัก

migrate dev

สร้าง migration (dev)

migrate deploy

รัน migration ใน production

db pull

ดึง schema จาก DB จริง

db push

sync schema → DB (ไม่มี history)

prisma db pull — ดึง schema จาก DB จริง

- ใช้กับระบบเดิมที่มี database อยู่แล้ว (legacy / brownfield)
- Prisma จะอ่าน table/column จริง แล้วสร้าง schema.prisma ให้เอง
- เหมาะสำหรับกรณีพัฒนาใหม่บนฐานข้อมูลที่องค์กรใช้อยู่



bash

```
# Introspect database ปัจจุบัน → สร้าง schema.prisma
npx prisma db pull
```

หลัง db pull จะเกิดอะไรขึ้น

- schema.prisma ถูก generate ใหม่อัตโนมัติ
- ทุก table ในฐานข้อมูล → ถูกแปลงเป็น model
- ทุก column → ถูกแปลงเป็น field พร้อม type
- ความสัมพันธ์ (foreign key) → ถูกแปลงเป็น @relation
- สิ่งที่เคยแก้ไขไว้ใน schema.prisma จะถูก overwrite

? ลบ model ที่ไม่ใช้ใน schema ได้ไหม

✓ ลบ model ได้

- ลบออกจาก schema.prisma ได้เลย
- Prisma Client จะไม่รู้จัก model นั้นอีก

! Database ไม่ถูกระบบ

- table จริงยังคงอยู่ในฐานข้อมูล
- ต้องใช้ migration แยกถ้าต้องการ drop

📌 สรุป

- schema = ใช้ใน code
- database = ยังอยู่เหมือนเดิม (จนกว่าจะ migrate drop)

ใช้ @@ignore — บอก Prisma ว่าอย่ายุ่งกับ table นี้

- ใช้กับ table เก่าที่ไม่ต้องการให้ Prisma Client จัดการ
- Prisma ยัง pull เข้ามาได้ แต่จะไม่ถูกใช้ในการ query
- ช่วยให้ schema สะอาด ไม่ error กับ table ที่ไม่มี @id

```
model OldTable {  
  id Int  
  
  @@ignore // Prisma Client จะข้าม model นี้  
}
```

⚠ ข้อควรระวังเมื่อใช้ db pull

schema ถูก overwrite

สิ่งที่แก้ด้วยมือใน schema.prisma จะหายไป

ต้องใช้ git เสมอ

commit ก่อน pull เพื่อ rollback ได้

ชื่อ relation อาจไม่สย

บางครั้งต้องแก้ชื่อ field หลัง pull

อย่า pull บน production โดยตรง

ใช้ dev DB ที่ clone มาจะปลอดภัยกว่า

เพิ่ม table ใหม่ใน schema

- เขียน model ใหม่ใน schema.prisma
- ยังไม่กระทบ database จนกว่าจะสั่ง migrate



prisma

```
model Product {  
  id    Int    @id @default(autoincrement())  
  name  String  
  price Float  
}
```

migrate หลังเพิ่ม table

- สั่ง migrate dev จะสร้าง SQL migration script ให้อัตโนมัติ
- ตั้งชื่อให้ชัดเจนเสมอ — อ่านประวัติได้ง่าย



bash

```
# เพิ่ม Product table เข้าสู่ database  
npx prisma migrate dev --name add_product
```

! จะเกิด Migration Conflict ไหม

✓ ไม่ conflict

- schema สอดคล้องกับ DB
- ทุกคนใช้ migration files ชุดเดียวกันจาก git

✗ จะ conflict เมื่อ

- แก้ database ด้วยมือ (manual SQL)
- ลบไฟล์ใน prisma/migrations/ โดยไม่ sync

กฎทอง: schema.prisma ↔ prisma/migrations ↔ database ต้องตรงกันเสมอ

ถ้าเคยแก้ DB ด้วยมือ — ต้องใช้ db pull + สร้าง migration ใหม่เพื่อให้ sync

Workflow ที่ถูกต้อง (Migration Flow)

กรณีมี DB อยู่แล้ว

1. db pull

2. แก้ schema ตามต้องการ

3. migrate dev

กรณีเริ่มใหม่

1. ออกแบบ schema.prisma

2. migrate dev

db push vs migrate — ต่างกันอย่างไร

หัวข้อ	db push	migrate dev / deploy
เหมาะกับ	prototype / dev เร็ว ๆ	ทีม / production
มี history?	❌ ไม่มี	✅ เก็บใน prisma/migrations
ใช้ใน prod?	❌ ไม่แนะนำ	✅ ใช้ migrate deploy
rollback	❌ ยาก	✅ ทำผ่าน git + re-migrate

Deployment — migrate deploy

- ใช้เมื่อ deploy ขึ้น staging / production
- ไม่สร้าง migration ใหม่ — แคร็นไฟล์ที่มีอยู่ใน prisma/migrations/
- ควรรันเป็นขั้นตอนแรกของ CI/CD ก่อนสตา์ท API



bash

```
# รันทุก migration ที่ยังไม่ถูกใช้กับ production DB  
npx prisma migrate deploy
```

Error Handling — try/catch

- ครอบ query ด้วย try/catch เสมอในโค้ด production
- จับ PrismaClientKnownRequestError เพื่อแยก error code เช่น P2002 (unique)



javascript

```
try {  
  await prisma.user.create({ data })  
} catch (err) {  
  if (err.code === "P2002") // email ซ้ำ  
    console.error(err)  
}
```

Performance — ทำให้ Query เร็วขึ้น

- ใช้ select เพื่อดึงเฉพาะ field ที่ใช้ — ลด payload
- สร้าง index บน field ที่ query บ่อยด้วย @@index
- หลีกเลี่ยง N+1 — ใช้ include แทน loop
- ใช้ pagination (take / skip หรือ cursor) เสมอในรายการใหญ่
- ใช้ connection pooling (PgBouncer / Prisma Accelerate)

Best Practices — Production Ready

ใช้ git เสมอ

commit schema.prisma และ prisma/migrations ทุกครั้ง

ไม่แก้ DB ด้วยมือ

ทุกการเปลี่ยน schema ต้องผ่าน migration

backup ก่อน migrate

โดยเฉพาะ production — สำรอง database ก่อนเสมอ

ตั้งชื่อ migration ชัดเจน

เช่น add_order_status, drop_legacy_log

แยก dev / prod DB

อย่ารัน migrate dev ชน production

Integration — ใช้งาน Express.js

- import prisma client เข้ามาใช้ใน route handler
- ใช้ async/await + return JSON กลับไปให้ client



javascript

```
// GET /users - คืนรายชื่อผู้ใช้ทั้งหมด
app.get("/users", async (req, res) => {
  const users = await prisma.user.findMany()
  res.json(users)
})
```

Prisma Studio — GUI จัดการข้อมูล

- GUI สำเร็จรูปสำหรับดู / แก้ไขข้อมูลในฐานข้อมูลผ่านเบราว์เซอร์
- เหมาะกับ developer / QA — ใช้แทน client DB เพิ่มเติมไม่ได้
- ไม่ควรเปิดเชื่อมตรงกับ production DB



bash

```
# เปิด Prisma Studio ที่ http://localhost:5555  
npx prisma studio
```

ตัวอย่างใช้งานจริง — User / Order / Approval

User

บัญชีผู้ใช้ขององค์กร

id
email
role

Order

คำสั่งซื้อ / request

id
userId
total
status

Approval

การอนุมัติตามลำดับชั้น

id
orderId
approverId
status

Relation: User 1-N Order 1-N Approval

สรุป — Prisma + Migration

Prisma = ORM Modern

- Type-safe, schema-first
- ลดเวลา dev และลด bug

Migration = หัวใจของระบบ

- Version-controlled schema
- Safe + repeatable deploy

db pull / push / migrate

- ใช้ให้ถูกสถานการณ์
- ป้องกัน schema conflict

Best Practices

- git + backup + CI migrate deploy
- ไม่แก้ DB ด้วยมือ

\$ _

Q & A

ขอบคุณที่ตั้งใจฟังครับ — เปิดถามคำถามได้เลย

Prisma ORM + Node.js • Production Ready